

第 2 次课 Git 版本选择操作清单

本清单用于帮助同学从课程仓库中选择第 2 次课（02-data-bss）的实验版本，并在该版本基础上开始自己的实验。每条命令都附了“这行在做什么”，文末 §6 专门收录了本次课同学实际踩过的坑和对应解法。

通用原则（换行符、WSL/Windows 分工等）见 docs/01/student_git_tag_guide.md，本篇只讲第 2 次课特有的部分 + 新增的踩坑记录，两篇建议都看。make / QEMU 必须在 **WSL Ubuntu** 里执行，PowerShell 里看到 ObjectNotFound: make 说明还没进 WSL。

0. 首次克隆仓库（已经 clone 过、直接从第 1 次课分支继续的同学可跳过）

建议统一在 **WSL Ubuntu** 里执行 `git clone`，不要在 Windows PowerShell 里 clone——原因见下方“踩坑提醒”，这是本次课 §6.1 那个“一大片文件 modified”问题的根源，从这一步就避免最省心。

```
git clone https://github.com/cxi218447-byte/miniOS.git
```

作用：把远程课程仓库完整拉到本地，包括 `master` 分支和所有课次的 tag。只需要执行一次；如果你上次课已经 clone 过，**不要重复 clone**，直接在原来的目录里继续（见 §1）。

```
cd miniOS
```

作用：进入刚 clone 下来的目录，后续所有 `git / make` 命令都要在这个目录里执行。

踩坑提醒：如果不确定自己上次是在 Windows 还是 WSL 里 clone 的，先执行一次 `git status`：

- 显示 `nothing to commit, working tree clean` → 没事，继续用。
- 显示一大片文件 `modified`（内容其实没改）→ 说明 clone 和现在操作的 git 不是同一边，直接跳到 §6.1 处理，**不要**在这个状态下继续做实验。

1. 本次课要用到的 tag

```
git fetch --tags
```

作用：从远程仓库把所有新增/更新的 tag 拉到本地（`master` 分支上老师发布的新内容，以及所有课次的验收点标签）。**不会改动你当前的工作区**，只是让本地 git 知道“远程现在有哪些版本可选”。

```
git tag
```

作用： 列出本地已知的所有 tag。第 2 次课只关心其中一个：

tag	含义
02-data-bss	第 2 次课正式验收点： <code>.data</code> / <code>.bss</code> 初始化 + C/汇编混合启动

(其余 `01-`、`03-` ... `12-` 开头的是别的课次，本次课不用管。)

2. 从 tag 新建自己的实验分支（不要直接在 tag 上改代码）

```
git switch -c my-02-lab 02-data-bss
```

逐段作用：

片段	含义
<code>git switch</code>	切换/创建分支的命令
<code>-c</code>	create，“创建一个新分支”，不加 <code>-c</code> 就只是切到已存在的分支
<code>my-02-lab</code>	新分支的名字（你自己起的，本地作业本）
<code>02-data-bss</code>	起点：从这个 tag 的内容出发建分支

参数顺序是“新分支名在前，起点在后”，两个都要给，缺一个就会出问题（具体见 §6.3，这是本次课最容易踩的坑）。

2.1 `my-02-lab` 为什么不在 GitHub 上？

名称	角色	谁创建	是否在 GitHub 上
<code>02-data-bss</code>	课程 tag，全班统一的验收快照	老师发布	是
<code>master</code>	当前已发布课次进度	老师维护	是
<code>my-02-lab</code>	你的本地作业本	你自己用 <code>-c</code> 创建	默认否，正常现象

不需要、也不建议 `git push -u origin my-02-lab` 到公共课程仓库。

2.2 确认自己做对了

```
git branch          # 当前分支前有 *, 应显示 * my-02-lab
```

作用：列出本地所有分支，* 标出当前所在分支。

```
git log --oneline -1
```

作用：查看当前分支最新一条提交的摘要，用来核对"我现在到底站在哪个版本上"。

```
git branch -r
```

作用：列出远程分支。正常情况下这里只有 `origin/master` 等，不会有 `origin/my-02-lab` ——这是设计如此，不是操作失败。

3. 编译运行

进入自己的实验分支后，在 **WSL Ubuntu** 里执行：

```
make clean  # 删除上次编译产物，避免残留文件干扰这次编译
make        # 按 Makefile 规则重新编译整个内核
make run    # 用 QEMU 启动编译好的镜像
```

预期输出（第 2 次课）参考 `docs/02/lab.md` 里给出的验收内容（`.data / .bss` 初始化相关提示），不再是第 1 次课单纯的“Hello miniOS”。

3.1 怎么退出 QEMU

`make run` 用的是 `-nographic` 模式（没有单独窗口，直接在当前终端里跑），退出方式：

```
Ctrl+A    然后单独按    X
```

不是同时按，是先按住 `Ctrl` 敲一下 `A` 松开，再单独按一下 `X`。终端出现类似 `QEMU: Terminated` 就说明退出成功，回到 `shell` 提示符。

踩坑提醒：

- `Ctrl+C` 在这个模式下**不一定好使**——`-nographic` 会把键盘输入转发给虚拟机里的串口/内核，`Ctrl+C` 可能被 miniOS 自己收到，而不是杀掉 QEMU 进程本身。
- 如果 `Ctrl+A X` 没反应，可以先 `Ctrl+A` 再按 `C` 进入 QEMU 自带的 monitor（会出现 `(qemu)` 提示符），输入 `quit` 回车退出：

```
Ctrl+A C  
(qemu) quit
```

- 实在卡死退不出，另开一个 WSL 终端窗口执行：

```
pkill qemu-system-loongarch64
```

强制结束进程。

4. 回到主分支

```
git switch master  
git pull  
git fetch --tags
```

- `git switch master`：切回主分支，**不会删除** `my-02-lab`，它还在，只是不在这个分支上了
- `git pull`：把远程 `master` 上老师发布的新内容同步到本地 `master`
- `git fetch --tags`：顺手再拉一次最新 tag，为下次课做准备

5. 做完实验如何清理（可选）

```
git switch master  
git branch -d my-02-lab
```

作用： `-d` 是"安全删除"，只有这个分支的内容已经合并/不会丢东西时才会成功；不删除也没关系，只是本机多一个分支名，不影响任何人。

6. 常见问题 / 踩坑记录（含本次课同学实际遇到的）

6.1 `git status` 显示一大片文件 `modified`，但没手动改过代码

现象：明明没改代码，`git status` 却把 `Makefile`、`kernel/main.c` 等一整批文件标成 `modified`；执行 `git checkout -- .` 之后还是没变。

原因：同一个仓库先后被 Windows 端 `git`（PowerShell / Windows Git Bash）和 WSL 端 `git` 操作过，两边对"要不要把换行符转成 CRLF"的设置（`core.autocrlf`）不一致，导致工作区文件的换行符和 WSL `git` 期待的不一样，被误判为"改过"。内容其实一个字节没变。

排查（确认真的只是换行符问题，不是真代码被改了）：

```
git diff Makefile | cat -A | head -5
```

如果能在行尾看到 `^M`，就是 CRLF 混入，不是真实代码差异。

修复（在 WSL 里执行）：

```
git config core.autocrlf input    # 以后只做 CRLF→LF 规范化，不再反向转换
git rm --cached -r . -q          # 清掉索引里按旧规则记录的内容
git checkout -- .                # 用当前设置重新签出工作区
git status                       # 应变回 nothing to commit, working tree clean
```

以后怎么避免：同一个仓库目录，固定只用一边的 **git 操作**——要么全程 WSL，要么全程 Windows，不要来回换。`git clone` 也建议 统一在 WSL 里做（见 §0），原因见 `docs/01/student_git_tag_guide.md §1`。

6.2 `git checkout -- .` 敲成了 `git checkout --`，看起来没反应

现象：想清掉工作区的改动，敲了 `git checkout --` 就回车，以为执行了，但 `git status` 完全没变化。

原因：`git checkout --` 末尾少了那个 `.`（或具体文件名）。`--` 后面没有给"要恢复哪些文件"（路径），Git 就当没有目标，**什么都不会做**，也不会报错，容易被误以为"已经执行成功但没用"。

正确写法：

```
git checkout -- .
```

`.` 表示"当前目录下所有文件"，这个点**不能省**。

6.3 `git switch -c 02-data-bss` 少写了自己的分支名

现象：想从 `02-data-bss` 这个 tag 建一个自己的实验分支，结果敲成：

```
git switch -c 02-data-bss
```

只给了一个参数。

原因：`git switch -c` 需要两个参数——`<新分支名>` `<起点>`。只给一个时，Git 会把它当成"新分支名"，**起点默认是当前 HEAD**（你敲这条命令时所在的位置），**不是真的从 tag `02-data-bss` 出发**。结果是多出一个**名字叫 `02-data-bss` 但内容其实是你之前所在分支**的本地分支——名字和内容对不上，很容易误导自己后续判断"我现在是不是 在验收点上"。

修复：

```
git branch                # 先看看现在都有哪些分支、自己在哪
git switch master         # 切到安全位置
git branch -d 02-data-bss # 删掉刚才建错的分支（提示无法删除就换 -D，前提是确认不要这个分支里的内容）
git switch -c my-02-lab 02-data-bss # 正确写法：新分支名在前，tag 起点在后
```

记忆口诀： `git switch -c` 后面永远是“**新名字 起点**”两个东西，起点是别人（tag/分支）已经有的名字，新名字是你自己起的、别人没用过的名字，**不要把 tag 名直接当成自己的分支名去用。**

6.4 `git switch -c my-02-lab 02-data-bss` 报错找不到 `02-data-bss`

先 `git fetch --tags` 更新本地 tag 列表，再重新执行 §2 的命令。

6.5 提示分支 `my-02-lab` 已经存在

换个名字（如 `my-02-lab-2`），或者直接切过去：

```
git switch my-02-lab
```

课次纯净说明

第 2 次课 tag（`02-data-bss`）在第 1 次课基础上加了 `.data / .bss` 初始化相关内容，**不包含**更后面课次的内容（无异常/系统调用等）。请始终从 `02-data-bss` 建分支做本次课实验，不要在混有更后续课次内容的 树上做本次课的实验。